

浙江大学实验报告

专业：自动化（控制）
姓名：_____
学号：_____
日期：2026.4.17
地点：东 3-411

课程名称：嵌入式系统 指导老师：彭勇刚、陆玲霞 实验类型：验证型
实验名称：实验二 汇编语言设计 成 绩：_____
签 名：_____

一、实验目的

- (1) 熟悉 Keil μ Vision 集成开发环境的使用方法，掌握汇编语言工程的建立、编译与仿真调试流程。
- (2) 掌握 8051 汇编语言中逻辑运算指令的使用方法。

二、实验设备

编号	仪器用具名称	主要参数
1	笔记本电脑	预装 Keil μ Vision 4 软件，支持 AT89C51 软件仿真环境

Table 1: 实验设备列表

三、实验原理

1. 逻辑运算指令

8051 提供 ANL（与）、ORL（或）、XRL（异或）、CPL（取反）四条基本逻辑运算指令，均以累加器 A 为目的操作数，支持寄存器、直接地址、间接地址及立即数寻址。

2. 求最值算法

将首个元素作为当前最小值和最大值的初值，依次遍历后续元素进行比对与更新。使用 SUBB 指令配合借位标志 CY 判断无符号数的大小：执行 CLR C；SUBB A, Rn 后，若产生了借位（CY=1），则表明 A 小于 Rn。

3. 多字节 BCD 码加减法

DAA 指令仅适用于加法运算后的二—十进制调整。由于 8051 指令集中无对应的减法调整指令，BCD 码减法必须转化为补码加法：先求取每字节减数的补码（低字节用 $9A_H$ 减，高字节用 99_H 减并保留借位），再将该补码与被减数相加，随后使用 DAA 指令对和进行十进制调整。

四、 预习要求

- (1) 预习 8051 逻辑运算指令和算术运算指令的寻址方式。
- (2) 理解 DAA 指令的功能及其限制（仅用于加法后调整）。
- (3) 了解 BCD 补码减法原理： $A - B = A + (9A_H - B)$ （低字节），高字节减数的补码为 $99_H - B$ （需带借位）。
- (4) 熟悉 Keil Memory 窗口的使用格式：D: 地址查看片内 RAM 直接寻址空间。

五、 实验内容

1. 实验操作方法和步骤

- (1) 打开 Keil μ Vision，新建工程，选择 Atmel AT89C51 芯片，拒绝添加启动文件。
- (2) 新建汇编源文件（.asm），保存后手动添加到工程的 Source Group 中。
- (3) 在 Options for Target 中选择软件仿真（Use Simulator）模式。
- (4) 编写程序，按 F7 编译，无误后按 Ctrl+F5 进入 Debug 模式。
- (5) 按 F5 全速运行至 SJMP \$，在 Memory 窗口输入 D: 地址查看结果。

2. 实验数据记录和处理

- (1) PART1 逻辑运算

题目：设 $A = 63_H$ ， $B = 82_H$ ， $C = C5_H$ ， $D = 36_H$ ，求

$$Y = \overline{A \oplus B \cdot C \cdot D + A}$$

完整程序代码：

```
1  MOV A, #63H ; 将 A 值给累加器 A
2  MOV R0, #36H ; 将 D 值给 R0
3  ORL A, R0; 或运算，存入 A
4  CPL A; 将 A 取反
5  MOV R1, A
6  MOV A, #82H
7  MOV R0, #0C5H
8  ANL A, R0; 与指令
9  CPL A
10 MOV R0, #63H
11 XRL A, R0; 异或
12 CPL A
13 ANL A, R1
14 SJMP $
```

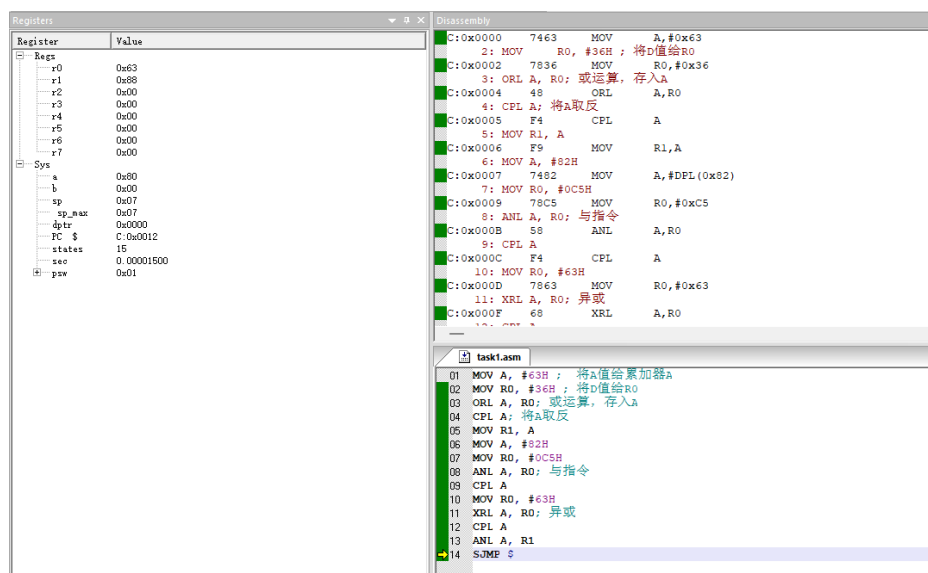


Figure 1: PART1 逻辑运算结果（80H）

运行结果：

在 Register 窗口查看 A 寄存器的值，发现是 80H。

结果验证：人工计算如下：

$$\begin{aligned}
 B \cdot C &= 82_H \text{ AND } C5_H = 80_H \\
 \overline{B \cdot C} &= 7F_H \\
 A \oplus \overline{B \cdot C} &= 63_H \text{ XOR } 7F_H = 1C_H \\
 \overline{A \oplus \overline{B \cdot C}} &= E3_H \\
 D + A &= 36_H \text{ OR } 63_H = 77_H \\
 \overline{D + A} &= 88_H \\
 Y &= E3_H \text{ AND } 88_H = 80_H
 \end{aligned}$$

答案为 $Y = 80_H$ ，与程序运行结果一致。

(2) PART2 求最小数和最大数

题目：10 个无符号数连续存放在以 20H 为起始地址的 RAM 中，找出最小值存入 30H，最大值存入 31H。

完整程序代码：

```
1      ORG 0000H
2      AJMP MAIN
3      ORG 0080H
4
5  MAIN:
6      ; ===== 写入测试数据 (20H~29H) =====
7      MOV R0, #20H
8      MOV @R0, #05H      ; 第 1 个数
9      INC R0
10     MOV @R0, #12H      ; 第 2 个数
11     INC R0
12     MOV @R0, #03H      ; 第 3 个数
13     INC R0
14     MOV @R0, #09H      ; 第 4 个数
15     INC R0
16     MOV @R0, #0FFH     ; 第 5 个数
17     INC R0
18     MOV @R0, #01H      ; 第 6 个数
19     INC R0
20     MOV @R0, #08H      ; 第 7 个数
21     INC R0
22     MOV @R0, #20H      ; 第 8 个数
23     INC R0
24     MOV @R0, #0AH      ; 第 9 个数
25     INC R0
26     MOV @R0, #07H      ; 第 10 个数
27
28     ; ===== 初始化：取第一个数为最小值和最大值 =====
29     MOV R0, #20H
30     MOV A, @R0
31     MOV R2, A          ; R2 = 当前最小值
32     MOV R3, A          ; R3 = 当前最大值
33     MOV R7, #09H       ; 循环计数器：还需比较 9 次
34
35  LOOP: INC R0           ; 指针后移
36        ACALL COMP      ; 调用比较子程序
37        DJNZ R7, LOOP    ; 循环 9 次
38        AJMP DONE
39
40     ; ===== 比较子程序 =====
41  COMP: CLR C
```

```

42      MOV    A, @R0
43      SUBB   A, R2          ; A - 最小值
44      JC     UPDATE_MIN    ; CY=1 说明 @R0 < R2, 更新最小值
45
46      CLR    C
47      MOV    A, @R0
48      SUBB   A, R3          ; A - 最大值
49      JNC    UPDATE_MAX    ; CY=0 说明 @R0 >= R3, 更新最大值
50      RET
51
52  UPDATE_MIN:
53      MOV    A, @R0
54      MOV    R2, A          ; 更新最小值
55      RET
56
57  UPDATE_MAX:
58      MOV    A, @R0
59      MOV    R3, A          ; 更新最大值
60      RET
61
62  DONE:    MOV    30H, R2    ; 存最小值
63          MOV    31H, R3    ; 存最大值
64          SJMP   $
65  END

```

运行结果：

Memory 窗口输入 D:0020H 查看原始数据，输入 D:0030H 查看结果。

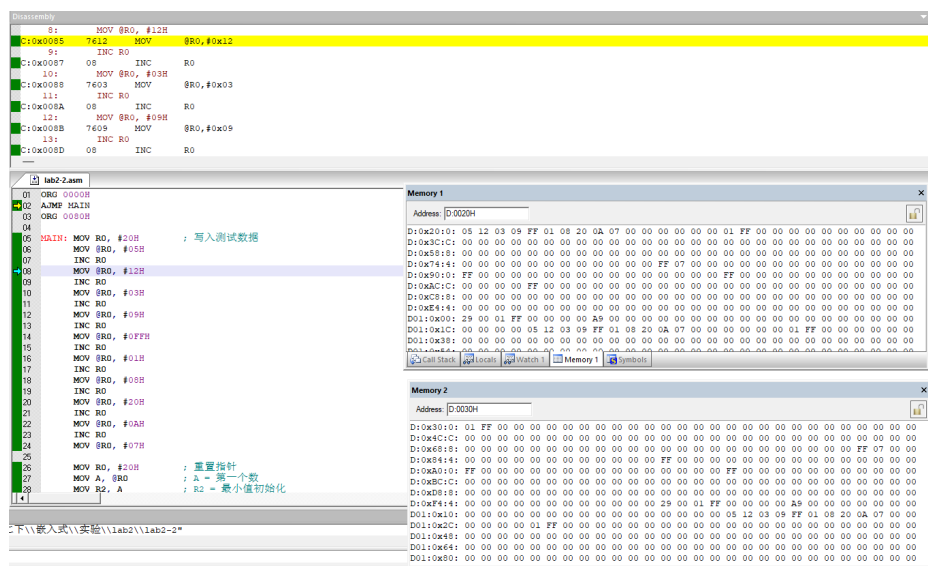


Figure 2: PART2 求最大值 Memory 窗口结果 (30H=01H, 31H=FFH)

测试数据为 05 12 03 09 FF 01 08 20 0A 07，预期最小值 = 01H，最大值 = FFH。

(3) PART3 十进制加减运算

题目：计算 $286729 + 652430 - 752196 = ?$ ，数据以 BCD 码低位在先格式存储于片内 RAM，结果存入 2DH~2FH。

数据存放格式：

数值	地址	内容（低位在先）
286729（被加数）	20H~22H	29H, 67H, 28H
652430（加数）	23H~25H	30H, 24H, 65H
752196（减数）	26H~28H	96H, 21H, 75H
结果（186963）	2DH~2FH	63H, 69H, 18H

Table 2: PART3 数据存放格式

完整程序代码：

```

1      ORG 0000H
2      AJMP MAIN
3      ORG 0100H
4
5  MAIN:
6      ; ===== 写入测试数据 =====
7      MOV R0, #20H
8      MOV @R0, #29H      ; 286729 低字节
9      INC R0
10     MOV @R0, #67H      ; 286729 中字节
11     INC R0
12     MOV @R0, #28H      ; 286729 高字节
13     INC R0
14     MOV @R0, #30H      ; 652430 低字节
15     INC R0
16     MOV @R0, #24H      ; 652430 中字节
17     INC R0
18     MOV @R0, #65H      ; 652430 高字节
19     INC R0
20     MOV @R0, #96H      ; 752196 低字节
21     INC R0
22     MOV @R0, #21H      ; 752196 中字节
23     INC R0
24     MOV @R0, #75H      ; 752196 高字节
25
26     ; ===== 第一步：286729 + 652430，结果暂存 2DH~2FH =====
27     CLR C
28     MOV R0, #20H
29     MOV R1, #23H
30
31     MOV A, @R0      ; 低字节相加
32     ADD A, @R1

```

```
33      DA    A
34      MOV   2DH, A
35
36      INC   R0
37      INC   R1
38      MOV   A, @R0          ; 中字节相加（带进位）
39      ADDC  A, @R1
40      DA    A
41      MOV   2EH, A
42
43      INC   R0
44      INC   R1
45      MOV   A, @R0          ; 高字节相加（带进位）
46      ADDC  A, @R1
47      DA    A
48      MOV   2FH, A
49
50      ; ===== 第二步：求 752196 的 BCD 补码，存入 31H~33H =====
51      ; 低字节补码：9AH - 减数（CLR C 保证无借位输入）
52      ; 中/高字节补码：99H - 减数（SUBB 会带上低位借位）
53      MOV   R0, #26H
54      MOV   R1, #31H
55      CLR   C
56      MOV   A, #9AH
57      SUBB  A, @R0          ; 9AH - 96H = 04H（低字节补码）
58      MOV   @R1, A
59
60      INC   R0
61      INC   R1
62      MOV   A, #99H
63      SUBB  A, @R0          ; 99H - 21H - CY（中字节补码）
64      MOV   @R1, A
65
66      INC   R0
67      INC   R1
68      MOV   A, #99H
69      SUBB  A, @R0          ; 99H - 75H - CY（高字节补码）
70      MOV   @R1, A
71
72      ; ===== 第三步：第一步结果 + BCD 补码 =====
73      CLR   C
74      MOV   R0, #2DH
75      MOV   R1, #31H
76
77      MOV   A, @R0          ; 低字节
78      ADDC  A, @R1
```

```

79      DA    A
80      MOV   @R0, A

81
82      INC   R0
83      INC   R1
84      MOV   A, @R0          ; 中字节（带进位）
85      ADDC  A, @R1
86      DA    A
87      MOV   @R0, A

88
89      INC   R0
90      INC   R1
91      MOV   A, @R0          ; 高字节（带进位）
92      ADDC  A, @R1
93      DA    A
94      MOV   @R0, A

95
96      SJMP  $
97      END

```

运行结果：

Memory 窗口输入 D:002DH 查看数据结果。

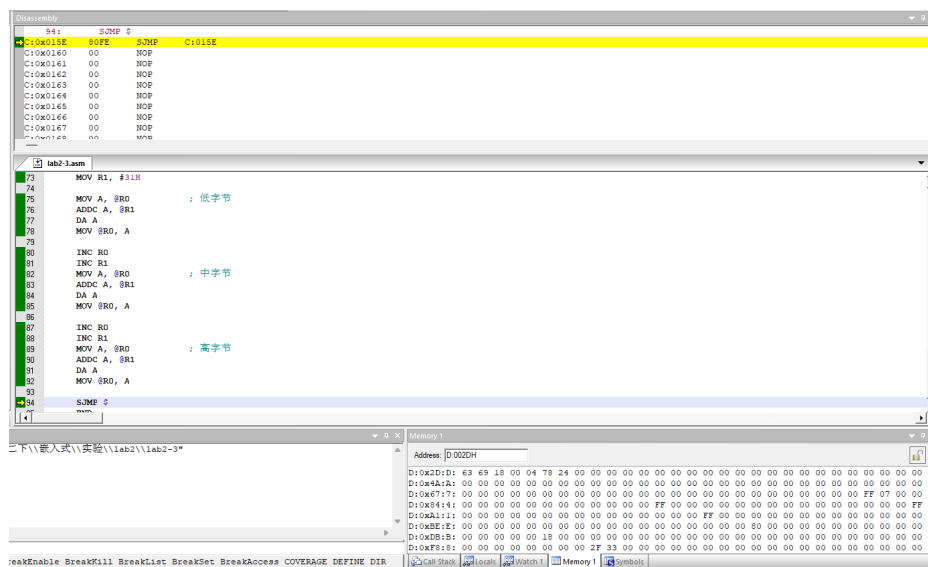


Figure 3: PART3 十进制加减运算结果（2DH=63H，2EH=69H，2FH=18H，即 186963）

六、实验总结

1. 实验结果与分析

三个实验均编译通过，Keil 仿真运行结果正确：

实验	题目	预期结果	实际结果
PART1	逻辑运算	30H = 80H	80H ✓
PART2	求最小值/最大值	30H=01H, 31H=FFH	01H / FFH ✓
PART3	BCD 加减运算	2DH~2FH = 63 69 18	63H 69H 18H ✓

Table 3: 实验结果汇总

2. 心得与体会

- (1) **程序越界故障与执行控制：**实验初期由于未在文件末尾设置合适的停机指令，CPU 在完成所有逻辑后继续取指，触发了 `access violation`（非法访问）错误。这使我认识到高级语言与底层的区别：汇编中的 `END` 仅是编译伪指令。通过在主体逻辑末端添加 `SJMP $` 构成死循环，成功解决了程序指针失控的问题，保障了系统运行的稳定性。
- (2) **寻址方式与编译异常处理：**在多字节数据处理时，我发现 Keil 环境对特定直接寻址指令序列（如 `ADD A, direct` 紧接 `DA A`）会引发异常编译报错。为解决这一问题，我将核心代码重构为使用 `@R0` 等寄存器间接寻址策略。这不仅迅速排除了编译器的解析故障，也使程序在处理连续存储空间时更加通用、精简。
- (3) **指令集限制下的算法变通：**在实现 BCD 码减法时，遇到了 8051 硬件指令集仅支持加法十进制调整（`DA A`）的平台限制。为了解决这一运算障碍，我利用构建补码（如加上 $99_H/9A_H$ ）的方法，将减法运算巧妙转化为加法运算再行调整。这种策略不仅解决了无法直接相减的错误，也让我深刻体会了计算机底层“以加代减”的逻辑基础。